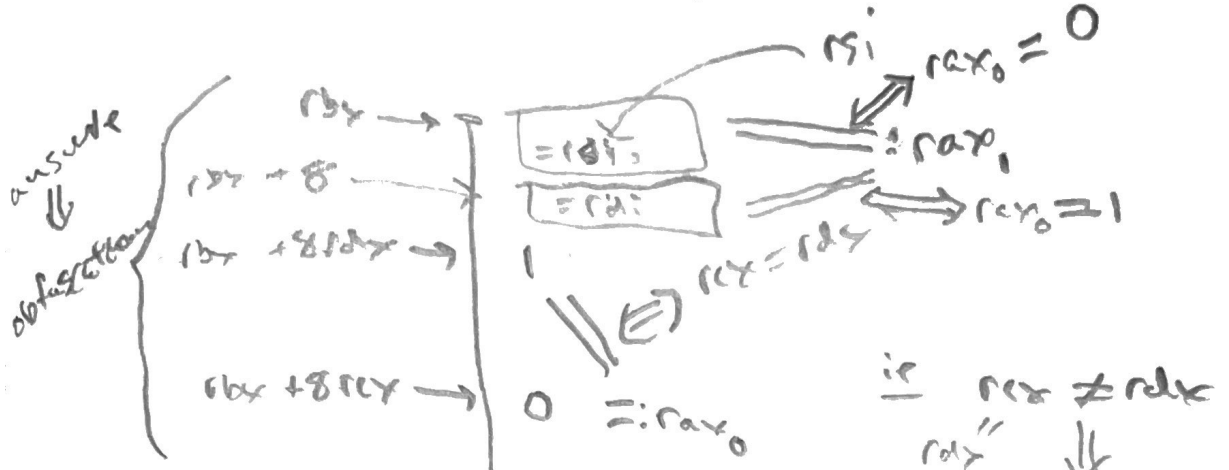
[0x2c]



branchless looking +saving of indices

ie rcx = rdx → Y → rdi, N → rsi

```
mov     qword [rbx + 8*rcx],0
mov     qword [rbx + 8*rdx],1
mov     rax,qword [rbx + 8*rcx]

mov     qword [rbx],rsi
mov     qword [rbx + 8],rdi
mov     rax,qword [rbx + 8*rax]
```

[rbx] := rsi

[rbx+8] := rdi

rax = [rbx + 8 {0,1}] ie :)

rax = (rcx == rdx? rdi : rsi)

0x2c

8 bytes (word = 2 bytes)

where to start

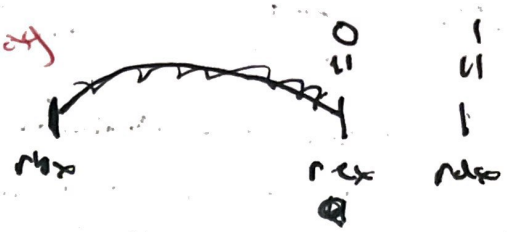
64 bits

size of each element

base + scale · index

which one (index of it)

```
mov  qword [rbx + 8·rcx], 0
mov  qword [rbx + 8·rdx], 1
```




```
mov  rax, qword [rbx + 8·rcx]
```

rcx := 0 (1 ⟺ rcx = rdx)

OR save state

```
mov  qword [rax], rsi
mov  qword [rbx + 8], rdi
mov  rcx, qword [rbx + 8·rcx]
```

~~set idx~~ idx

~~dont idx~~ ?

obfuscation technique

~~rcx ≟ {0, 1}~~

set the next (or just) value ~~to~~ to rax

IF rcx = 0 + rdx = 1 ⟹ can move rsi, rdi to wherever you want

ie malware gets access to code changes if looks innocent + is hard to see ?

just a lookup table

(0 or 1) ⟶ (rsi or rdi)

rsx-offset ∈ rbx

first try